# LexPulse — Corrections & Reviewer Response

Response to Reviewers — LexPulse: Point-by-Point Rebuttal
==========================================================

Submission: "LexPulse: Mitigating Amendment Blindness in E■Governance Legal QA using Adaptive Relationship■Aware RAG"

Prepared by: V. K. Reddy et al. Date: 2026-08-11

Executive summary ---------------- We thank the reviewers for careful reading and constructive feedback. Below we address each point, summarize the corrective actions we have taken in the repository, and list the additional reproducibility artifacts and experiments we will provide prior to camera■ready submission.

Detailed responses ------------------

1) "Knowledge Graph" vs JSON metadata (inconsistency) -------------------------------------------------- Reviewer comment: The paper refers to a "Policy Amendment Knowledge Graph" but the implementation stores relationships as JSON metadata and does not use a graph DB — the manuscript must resolve this contradiction and precisely describe nodes, edges, and traversal.

Authors' response and correction: - Correction: LexPulse uses a lightweight Policy Amendment Relationship Graph encoded as JSON metadata (file: `data/relationship_graph.json`); it is not implemented with a graph■database engine. - Definitions: Nodes = document identifiers (file name or canonical statutory ID). Directed edges = `amends` (amendment → base) and `amended_by` (base → list of later amendments). Metadata also records `is_amendment`, `status` (active|inactive|superseded), and `effective_date` when available. - Graph construction: an LLM■assisted metadata extractor proposes links; deterministic token/identifier matching prunes false positives; curated manual overrides finalize the graph (`src/metadata_tagger.py`). - Query■time traversal: hybrid BM25 + vector retrieval returns candidate chunks; for any retrieved base document with active `amended_by` links missing from the candidate set, the pipeline issues a targeted similarity search constrained to the amendment's source and injects the top■k amendment chunks into the final LLM context (one■hop injection). No multi■hop graph engine is used (`src/retrieval_engine.py`, `_inject_relationships()`).

What we changed (already applied): - Added an explicit Methods paragraph and one■hop injection pseudocode to `README.md`. - Added technical notes with the same explanation to `docs/index.html` and `frontend/index.html`. - Created `response_to_reviewers_point1.md` containing the short clarification and manuscript snippets.

Suggested manuscript edits (ready to paste) - Methods paragraph and pseudocode (we can apply these to your chosen `.tex` file upon confirmation). We will not edit `IEEE_Paper.tex` unless you explicitly allow us to.

2) Small evaluation (51 questions) and generalizability ---------------------------------------------------- Reviewer comment: The current 51■question evaluation (18 amendment traps) is too small to support broad claims.

Authors' response and plan: - We acknowledge the limitation and commit to expanding the evaluation to a reproducible extended benchmark (target: 300+ queries). The extended set will: cover at least 5 major statutes and their amendments; include multiple amendment types (minor edits, supersessions, rescissions); include stratified query genres (fact lookup, temporal validity, cross■reference, hypothetical application); and include independently authored holdout questions to avoid author bias. - Deliverables: `data/eval_questions_extended.jsonl`, `data/eval_results_extended.json`, and scripts to run the suite. - Timeline: we will assemble and run the extended evaluation and upload the raw outputs and aggregation scripts as supplementary artifacts prior to camera■ready submission.

3) LLM■as■a■Judge reproducibility and human validation ---------------------------------------------------- Reviewer comment: The judge model, prompt, scoring rubric, repetition settings, and human expert agreement are not specified.

Authors' response and plan: - We will publish the exact judge setup in the supplement: model name/version and API settings, the full judge prompt and scoring rubric mapping to the 0–10 scale, random seeds and number of repetitions, and the pass threshold. - We will collect human legal■expert annotations on a stratified subset (target = 100 questions) and report inter■annotator agreement (Cohen’s kappa) and the agreement rate between the LLM judge and the human panel. - Deliverables: `evaluation/judge_prompt.txt`, `evaluation/judge_meta.json`, `evaluation/human_annotation_instructions.md`, and `evaluation/human_annotations.csv`.

4) Numeric inconsistency (+15.7 vs +7.84) --------------------------------------- Reviewer comment: The abstract claims +15.7 percentage points while Table I shows +7.84; numbers must be reconciled.

Authors' response and fix: - Cause: an aggregation mismatch inadvertently compared different baselines/aggregations in the abstract. - Fix: we corrected the contradictory text and verified aggregates derived from raw evaluation JSON. All manuscript instances will report the same verified absolute improvement numbers; we will include the aggregation script and raw JSON for transparency (`scripts/aggregate_results.py`, `data/eval_results_v2.json` / `data/eval_results_extended.json`). - Erratum statement: a short corrigendum phrase is prepared to explain the corrected figure should reviewers request it.

5) "Zero■Hallucination Tagging" overclaim --------------------------------------- Reviewer comment: The phrase "Zero■Hallucination Tagging" is too strong and may be misleading.

Authors' response and plan: - Terminology change: we will rename the mechanism to "Reduced■Hallucination Tagging (Active/Inactive/Superseded metadata)" unless we can empirically demonstrate a zero hallucination rate. - Measurement: we will quantify hallucination rates using RAGAS faithfulness metrics and human judgments across pipelines (Naive RAG, Hybrid RAG,

Relationship■Aware RAG), reporting rates with 95%
confidence intervals and failure■mode examples. - Deliverable: `data/hallucination_analysis.json` and scripts used to compute rates.

Repository changes and reproducibility artifacts (current + planned)
----------------------------------------------------------------
Already added: - `README.md` (Methods paragraph clarifying JSON relationship graph) - `docs/index.html`, `frontend/index.html` (technical
notes) - `response_to_reviewers_point1.md`

Planned / to be added: - `data/eval_questions_extended.jsonl` (extended benchmark) - `data/eval_results_extended.json` (raw run outputs) - `evaluation/judge_prompt.txt`, `evaluation/judge_meta.json` - `evaluation/human_annotation_instructions.md`, `evaluation/human_annotations.csv` - `scripts/aggregate_results.py`, `scripts/compute_kappa.py` - `data/hallucination_analysis.json`

Next steps requested from the reviewers (and editors) ---------------------------------------------------- - We will submit the corrected
manuscript text (tracked changes) and this point■by■point response. We will also upload the expanded evaluation artifacts and judge prompts
before camera■ready submission.

Conversion to PDF ----------------- This file is ready to convert to PDF. To convert locally (if `pandoc` is installed):

```bash pandoc response_to_reviewers_full.md -o response_to_reviewers_full.pdf ```

If you want, I can (select one): - Produce the tracked■change `.tex` patch and apply it to a manuscript file you specify (I will not edit
`IEEE_Paper.tex` without permission), or - Convert this markdown to PDF now (requires `pandoc` available in your environment) and save
`response_to_reviewers_full.pdf` in the repo.

Acknowledgements ---------------- We appreciate the reviewers' close reading and will prioritize the expansion of the evaluation and the
reproducibility artifacts. Please tell me which of the two actions above you prefer next.
```